

1 Two-Server Private Inference Model

We now apply secret sharing to a simple privacy-preserving machine learning problem. Consider a model that computes $f(\mathbf{x}) = \mathbf{W}^t \mathbf{x} + \mathbf{b}$, where $\mathbf{W}$ and $\mathbf{b}$ are the model parameters and $\mathbf{x}$ is the user's private input. The goal is to evaluate the model without revealing the user's input to the servers or the model parameters to the user.

1.1 The Inference Setting

Suppose a company owns the trained model $(\mathbf{W}, \mathbf{b})$, while a user holds a private input $\mathbf{x}$. The user wants to obtain the prediction $f(\mathbf{x})$ without revealing $\mathbf{x}$ to the company. At the same time, the company does not want to reveal its model parameters to the user.

A straightforward deployment does not provide both forms of privacy. If the user sends $\mathbf{x}$ directly to the server, the server learns the private input. If the model parameters are sent to the user, the user obtains the company's model. We therefore need a way to perform the inference while keeping both the input and the model private.

1.2 Why Two Servers?

We use two non-colluding servers, denoted by S_0 and S_1 . Non-colluding means that the servers follow the protocol but do not share their internal information with each other.

The main idea is to represent every private value using two additive shares. For a secret s , the two shares s_0 and s_1 satisfy $s = s_0 + s_1 \pmod{q}$. Individually, neither share reveals the secret.

Thus, the model owner can share $\mathbf{W}$ and $\mathbf{b}$ between the two servers, while the user can independently share $\mathbf{x}$. The servers then perform the inference directly on these shares.

1.3 The Basic Approach

The entire computation is kept in shared form.

1. **Share the model.** The model owner generates additive shares of $\mathbf{W}$ and $\mathbf{b}$ and gives one share to each server.
2. **Share the input.** The user generates additive shares $\mathbf{x}_0$ and $\mathbf{x}_1$ of $\mathbf{x}$ and sends $\mathbf{x}_i$ to S_i . Thus, neither server receives $\mathbf{x}$ itself.
3. **Compute on shares.** The servers jointly evaluate $f(\mathbf{x})$ using their shares. Additions can be performed locally, while multiplications require a secure multiplication protocol.
4. **Reconstruct the output.** At the end, the servers hold shares of the prediction. These shares are sent to the user, who combines them to obtain $f(\mathbf{x})$.

The central question is therefore how the servers can perform the required multiplications without revealing their secret shares.

1.4 The Key Obstacle: Multiplication

Additive secret sharing makes linear computation simple. If the servers hold shares of two secrets a and b , each server can add its shares locally to obtain shares of $a + b$. Similarly, multiplication by a public constant can also be performed locally.

The model $f(\mathbf{x}) = \mathbf{W}^t \mathbf{x} + \mathbf{b}$, however, contains products between secret values. Both $\mathbf{W}$ and $\mathbf{x}$ are private and secret-shared, so the servers must securely multiply their shares.

For example, if a and b are secret-shared, directly multiplying the shares produces terms involving shares held by different servers. These cross terms cannot be computed locally without communication, and simply exchanging the shares would reveal private information. We therefore need a dedicated protocol for multiplying secret-shared values.

2 Beaver Triples: Secure Multiplication

Beaver triples were introduced by Beaver [1] to move the cost of multiplication into an offline phase, building on the general secret-sharing-based MPC framework of Ben-Or, Goldwasser, and Wigderson [?].

Consider two non-colluding servers S_1 and S_2 . Suppose the servers hold additive shares of two private secrets s_1 and s_2 . The sharing is represented as $s_1 = x_1 + y_1$ and $s_2 = x_2 + y_2 \pmod{q}$.

Thus, S_1 holds (x_1, x_2) , while S_2 holds (y_1, y_2) . The servers want to compute additive shares of the product $s_1 s_2$ without revealing s_1 or s_2 .

We assume that the two servers **do not collude**. Each server follows the protocol but does not share its private information with the other server.

Goal The goal is to compute shares of $s_1 s_2$ from the existing shares of s_1 and s_2 . At the end of the protocol, S_1 and S_2 should hold values z_1 and z_2 such that $z_1 + z_2 = s_1 s_2 \pmod{q}$, while neither server learns the complete values of s_1 or s_2 .

2.1 Design Idea

The main difficulty is that direct multiplication of the shares produces terms involving values held by different servers. Beaver triples provide correlated randomness that allows these cross-server products to be computed securely.

A Beaver triple consists of three random values (a, b, c) satisfying $c = ab \pmod{q}$. These values are also additively shared between the two servers. Thus, S_1 holds (a_1, b_1, c_1) and S_2 holds (a_2, b_2, c_2) , where $a = a_1 + a_2$, $b = b_1 + b_2$, and $c = c_1 + c_2$.

The triple is generated during an offline preprocessing phase, before s_1 and s_2 are known, and is used only once. The key idea is to mask the two secret values using the random values a and b . The servers reveal only the masked differences, which do not reveal the underlying secrets.

2.2 Protocol

Input: S_1 holds (x_1, x_2) and S_2 holds (y_1, y_2) , where $s_1 = x_1 + y_1$ and $s_2 = x_2 + y_2$. The servers also hold shares of a Beaver triple (a, b, c) satisfying $c = ab \pmod{q}$.

Step 1: Compute masked differences. Each server locally computes its shares of the differences $d = s_1 - a$ and $e = s_2 - b$. Thus, S_1 computes $d_1 = x_1 - a_1$ and $e_1 = x_2 - b_1$, while S_2 computes $d_2 = y_1 - a_2$ and $e_2 = y_2 - b_2$.

Step 2: Open the masked differences. The servers exchange their shares and reconstruct $d = d_1 + d_2$ and $e = e_1 + e_2$. Hence, $d = s_1 - a$ and $e = s_2 - b$ become public.

Since a and b are random values unknown to the servers, the opened values d and e do not reveal s_1 or s_2 .

Step 3: Compute shares of the product. The servers compute their output shares as $z_1 = c_1 + db_1 + ea_1$ and $z_2 = c_2 + db_2 + ea_2 + de$.

The final term de is added by only one server (See Figure 2.2).

2.3 Correctness

Since $d = s_1 - a$ and $e = s_2 - b$, we have $s_1 = a + d$ and $s_2 = b + e$. Therefore, $s_1 s_2 = (a + d)(b + e) = ab + db + ea + de$.

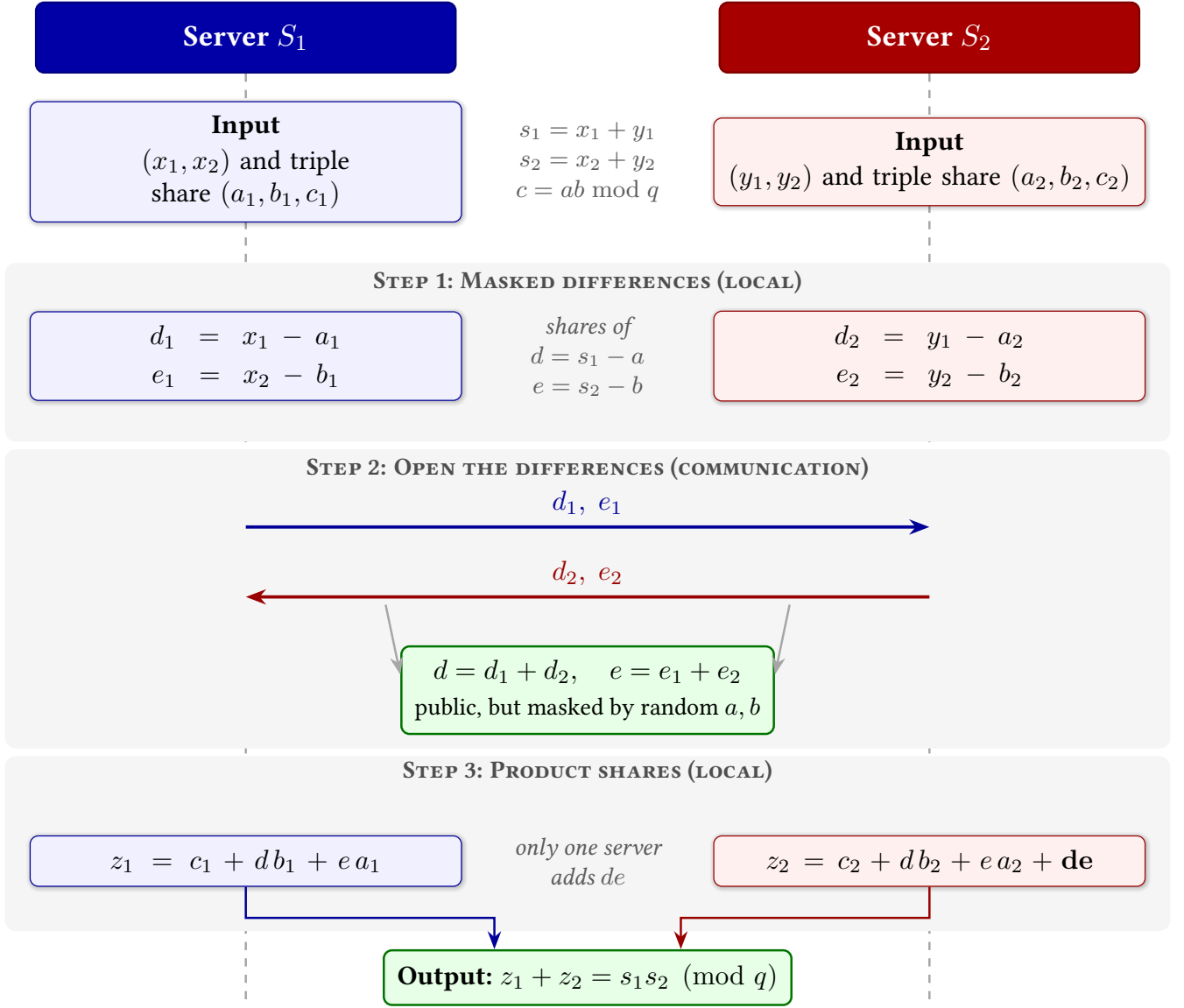


Figure 1: Two-server multiplication of shared secrets s_1 and s_2 using a Beaver triple.

Because the Beaver triple satisfies $c = ab$, we obtain $s_1 s_2 = c + db + ea + de$.

On the other hand, adding the two output shares gives $z_1 + z_2 = c + db + ea + de$.

Therefore, $z_1 + z_2 = s_1 s_2 \pmod{q}$.

Hence, (z_1, z_2) is a valid additive sharing of the product $s_1 s_2$.

2.4 Privacy Intuition

The only values revealed during the protocol are $d = s_1 - a$ and $e = s_2 - b$. Since a and b are random and unknown to either server, these values act as one-time masks for s_1 and s_2 . Consequently, an individual server cannot recover either secret from the opened values.

The Beaver triple therefore allows the servers to transform shares of s_1 and s_2 into shares of $s_1 s_2$ without revealing the underlying secrets.

3 Completing the Private Inference Model

We can now combine secret sharing and Beaver multiplication to construct the complete two-server inference procedure. See Fig 3.

1. **Setup phase.** The model owner secret-shares the model parameters $\mathbf{W}$ and $\mathbf{b}$ between S_0 and S_1 . The servers also receive a collection of Beaver triples sufficient for the multiplications required during inference.
2. **Online phase.** The user first secret-shares $\mathbf{x}$ and sends one share to each server. The servers then evaluate the model using their shares. For the linear model $f(\mathbf{x}) = \mathbf{W}^t \mathbf{x} + \mathbf{b}$, the servers securely compute the product $\mathbf{W}^t \mathbf{x}$ using Beaver multiplication. The addition of the bias $\mathbf{b}$ is local because addition of secret shares requires no communication.

After the computation, the servers hold shares of the prediction y . They send their output shares to the user, who combines them to recover $y = f(\mathbf{x})$.

Privacy. At no point does either server obtain the complete input $\mathbf{x}$ or the complete model parameters $(\mathbf{W}, \mathbf{b})$. Each server sees only its own shares and the masked values exchanged during Beaver multiplication. Under the non-collusion assumption, these individual views do not reveal the underlying private values.

Summary. The main idea is simple: secret sharing allows the servers to perform additions and other linear operations directly on shares, while Beaver triples provide a way to perform multiplication securely. Together, they allow the servers to evaluate $f(\mathbf{x}) = \mathbf{W}^t \mathbf{x} + \mathbf{b}$ while keeping both the user's input and the model parameters private.

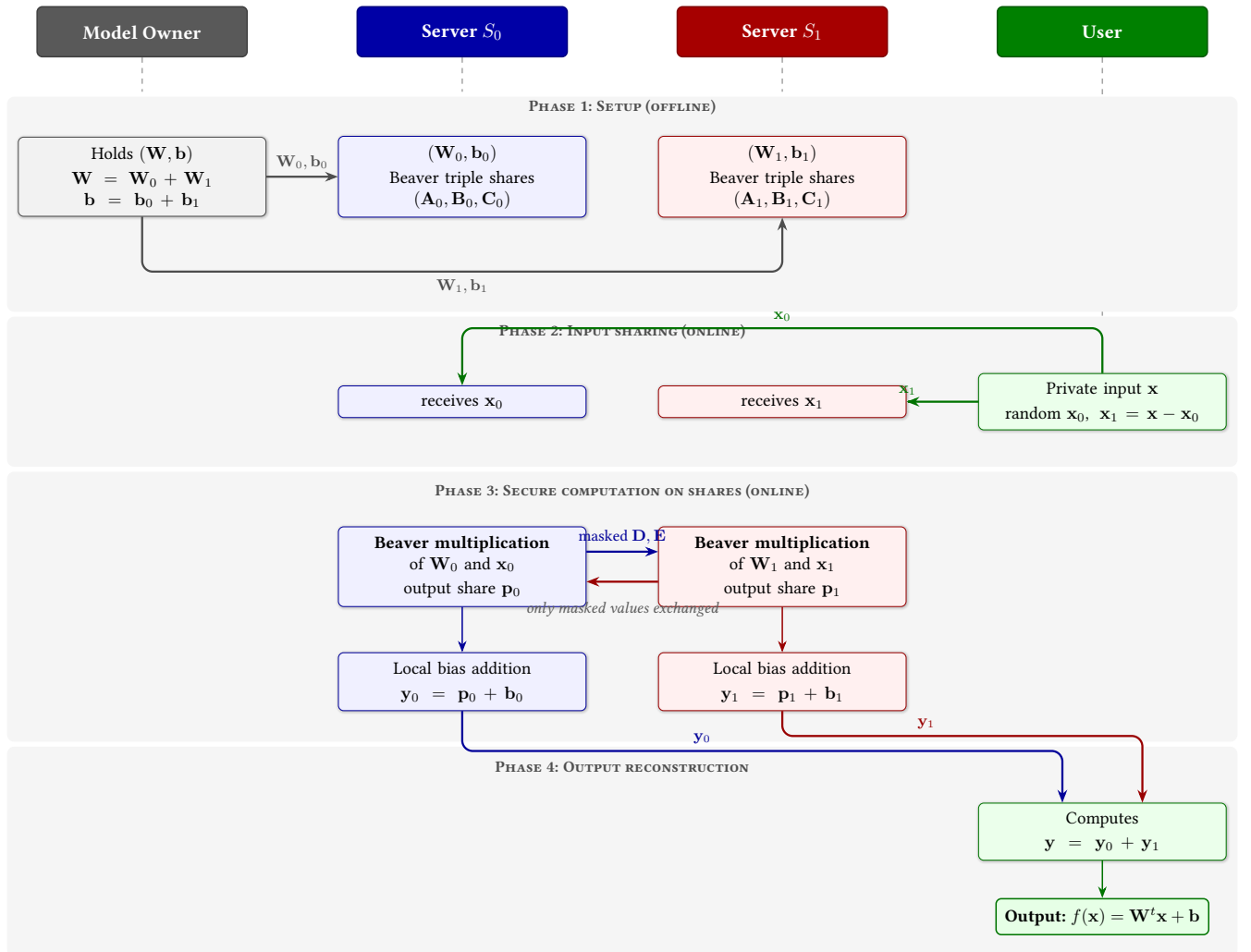


Figure 2: Interactions between parties in Two-server private inference for the linear model $f(\mathbf{x}) = \mathbf{W}^t \mathbf{x} + \mathbf{b}$.

References

- [1] D. Beaver, "Efficient Multiparty Protocols Using Circuit Randomization," in *Advances in Cryptology – CRYPTO '91*, Lecture Notes in Computer Science, vol. 576, pp. 420–432, Springer, 1992.
- [2] W. Du and M. J. Atallah, "Privacy-Preserving Cooperative Scientific Computations," in *Proceedings of the 14th IEEE Computer Security Foundations Workshop (CSFW)*, pp. 273–282, 2001.